

Customer Service Hours

Mon – Fri 8:30 AM – 4:30 PM MST

Upcoming Closures

The support team will be out of office on Monday, the 25th of May. We will return on Tuesday, the 26th of May, to normal business hours.

OBDLink® CX Adapter Notes

Modified on: Wed, 16 Jul, 2025 at 12:10 PM

If you're a developer who wants to create an application that uses the OBDLink® CX adapter, the information in this article is for you. This article isn't an all-inclusive developer's guide for creating and coding an application. It contains specific product information about the OBDLink CX adapter that you'll need when creating an OBD-based app.

This article contains the following sections:

- [OBDLink CX overview](#)
- [Technical specifications](#)
- [BLE-specific UUID information](#)
- [Maximum Transmission Unit \(MTU\) overview](#)
- [Android](#)
- [iOS](#)
- [Example code](#)
- [Learn more](#)
- [Need more help?](#)

OBDLink CX overview

The OBDLink CX adapter is a wireless OBD adapter that transforms any iPhone, iPad, or Android device into a professional-grade diagnostic scan tool, trip computer, and real-time performance monitor capable of accessing manufacturer-specific vehicle data.

The CX model is the only device in the OBDLink product lineup to use Bluetooth Low Energy (BLE).

OBDLink CX is optimized for maximum stability when coding. This means fewer dropped packets and data corruption than the competition. You get peace of mind that OBDLink CX won't brick your ECU, resulting in costly dealership repairs.

OBDLink CX has a low profile design and incorporates features not found in competing adapters, like overvoltage/load dump protection and ultra-low-current sleep mode, which allow you to leave it plugged in all the time. For added peace of mind, OBDLink CX comes with lifetime firmware upgrades.

Technical specifications

This section describes technical details for the OBDLink CX adapter.

- Supported Protocol Pins:
 - HS-CAN (pins 6 & 14)
 - ISO/KWP (pins 7 & 15)
- Supported Protocol Standards:
 - ISO 15765 (CAN for vehicle diagnostics)
 - SAE J1939 (heavy duty vehicles)
 - ISO 11898 (raw CAN)
 - ISO 9141 (Asian, European, Chrysler vehicles)
 - ISO 14230 (Keyword Protocol 2000)
- Bluetooth version:
 - Bluetooth Low Energy v5.1
 - No Classic Bluetooth support
- Operating current:
 - 55 mA
- Sleep current:
 - <2 mA
- OVP/load dump protection:
 - Up to 100 Volts
- Wireless security:
 - Bonding enabled for the first 5 minutes after power on
 - PIN for older BLE standards is 123456
 - The device always uses the built-in BLE encrypted communications
- LEDs:
 - Power/status
 - OBD Activity
 - Link status
- Dimensions (mm):
 - 46W x 32L x 22H

BLE-specific UUID information

This section describes certain Universally Unique Identifiers (UUIDs) that are used by the OBDLink CX adapter. (A UUID is a 128-bit label used for information.)

Service - device information

0000180A-0000-1000-8000-00805F9B34FB (180A)

- Characteristic - Manufacturer Name
 - OBD Solutions, LLC
- Characteristic - Model Number
 - OBDLink CX
- Characteristic - Firmware Revision

Note: This is the firmware version running on the STN IC. It is automatically updated after a firmware upload. This value is typically useful for application developers.

- xx.xx.xx (for example, 5.6.19)

- Characteristic - Software Revision

Note: This is the Bluetooth modem's firmware version. Application developers typically do not need this value.

- xx.xx.xx (for example, 1.1.3)

Service - custom UART

0000FFF0-0000-1000-8000-00805F9B34FB (FFF0)

- Characteristic - Notification
0000FFF1-0000-1000-8000-00805F9B34FB (FFF1)
- Characteristic - Write, Write without Response
0000FFF2-0000-1000-8000-00805F9B34FB (FFF2)

Service - for internal use only

0000FEF5-0000-1000-8000-00805F9B34FB (FEF5)

Maximum Transmission Unit (MTU) overview

Maximum Transmission Unit (MTU) is a measurement in bytes of the largest data packets that an Internet-connected device can accept. The largest MTU value for the OBDLink CX adapter is 247 bytes. In order to communicate with it, the MTU on the host device must be set properly.

Warning: If you are having issues sending more than 20 bytes, then your app is likely not handling the MTU size properly. The maximum possible MTU size varies between host devices; it is normal for it to be less than 247 bytes. Data being sent over BLE (i.e. ST commands, data, etc.) that exceeds the actual MTU size of the connection will fail to transmit if the app attempts to send it in one chunk.

To resolve this limitation, you'll need to develop a method to chunk the data down to the (MTU size minus overhead) to send long messages.

Note: The OBDLink CX adapter does not support QueuedWrites.

Android

This section describes what you need to do for an Android application.

Pair with OBDLink CX (Android)

This section contains a high level view of how the pairing process works.

1. Scan for BLE devices.

This is an OS-level request. For details, see the Android developer documentation, specifically the [Bluetooth Low Energy](https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview) (<https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview>) section.

2. Connect.

3. Discover Services.

This is an OS-level request. For details, see the Android developer documentation, specifically the [Use network service discovery](https://developer.android.com/develop/connectivity/wifi/use-nsd) (<https://developer.android.com/develop/connectivity/wifi/use-nsd>) (<https://developer.android.com/>

[develop/connectivity/wifi](#))section.

4. Prompt the user to pair by either subscribing to characteristic FFF1 or writing to characteristic FFF2.

MTU (Android)

MTU recommendations for Android devices:

- Default MTU size is 23 bytes
- When sending using `write.default()`, there is a 3 byte overhead. When using `write.withoutResponse()`, there is a 12 byte overhead. So if MTU size is 23, only 20 bytes are available when using `write.default()`.
- To get a higher MTU size:
 - Request MTU max size of 512 bytes, using `BluetoothGatt.requestMTUSize(512)`
 - Wait for the negotiated result in the `onMtuChanged()` callback (MTU is passed in as a parameter)

Learn more: See [requestMtu](https://developer.android.com/reference/android/bluetooth/BluetoothGatt#requestMtu(int)) ([https://developer.android.com/reference/android/bluetooth/BluetoothGatt#requestMtu\(int\)](https://developer.android.com/reference/android/bluetooth/BluetoothGatt#requestMtu(int))) in the Android developer documentation.

iOS

This section describes what you need to do for an iOS application.

Pair with OBDLink CX (iOS)

This section contains a high level view of how the pairing process works.

1. Scan for BLE devices.
This is an OS-level request. For details, see the Apple developer documentation, specifically the [Core Bluetooth](https://developer.apple.com/documentation/corebluetooth/) (<https://developer.apple.com/documentation/corebluetooth/>) section.
2. Connect.
3. Discover Services.
This is an OS-level request. For details, see the [Apple developer documentation](https://developer.apple.com/documentation) (<https://developer.apple.com/documentation>).
4. Prompt the user to pair by either subscribing to characteristic FFF1 or writing to characteristic FFF2.

MTU (iOS)

MTU recommendations for iOS devices:

- No default MTU size, it is negotiated automatically to the highest value supported by both host and device
- Maximum MTU size:
 - iOS 8 - 135
 - iOS 9 - 158
 - iOS 10 - 185
- (iOS 9+) You can get the negotiated MTU size by using `CBPeripheral.maximumWriteValueLength(for:.withoutResponse)`

Learn more: See [maximumWriteValueLength](https://developer.apple.com/documentation/corebluetooth/cbperipheral/) (<https://developer.apple.com/documentation/corebluetooth/cbperipheral/>)

`maximumwritevalulength(for:)` in the Apple developer documentation.

Note: iOS seems to have an issue where `CBCentral.maximumUpdateValueLength` and `CBPeripheral.maximumWriteValueLength(for:.withResponse)` return 512 regardless of negotiated MTU size. You can learn more at StackOverflow, specifically this post: [Change iOS BLE MTU size to 512 \(SWIFT\)](https://stackoverflow.com/questions/68375433/change-ios-ble-mtu-size-to-512-swift) (<https://stackoverflow.com/questions/68375433/change-ios-ble-mtu-size-to-512-swift>)

Example code

This section is generic for both Android and iOS applications.

The OBDLink CX adapter uses a custom BLE service as a serial port, so we need to provide a serial-like interface for application code. This includes abstracting over the asynchronous nature of BLE, as well as dealing with the maximum buffer size and message acknowledgement.

It is also important to plan for communication errors, and to handle timeouts and cancellations gracefully. We recommend that you use a modern concurrency framework or library to help manage these issues, because doing so manually can be tedious and error-prone.

The code in the following sections attempts to be framework and language-agnostic, but it uses C++ syntax and a fictitious BLE API, so it's up to you to find the equivalent functionality in your respective language. Before reading and writing, we assume the device has been connected to, paired with, and its services and characteristics have been discovered.

Writing

In order to write data to our device, there are two things you have to keep in mind. First, the maximum buffer size dictated by the negotiated MTU size (see [Maximum Transmission Unit \(MTU\) overview](#) above), and second, the write response to each write request.

Note: As shown below, it is important to wait for each write response before sending another write request. Typically, the write response event comes in asynchronous, so it is up to you to communicate this to the write routine.

```
class MyBleHandler: BLEHandler
{
    // Any concurrency primitive that allows you to send a "signal" can work here
    Semaphore writeSemaphore;

public:
    // This overrides the parent class BLEHandler's writeResponse event handler
    void writeResponse() override
    {
        writeSemaphore.signal();
    }

    // Typically, calling wait on a semaphore frees up the thread to do other tasks until the
    // signal is received
    // This is preferable to a loop repeatedly checking and unnecessarily burning cycles
    void waitForWriteResponse()
    {
        writeSemaphore.wait();
    }

    // Alternatively take a timeout and either throw an exception or return some sort of error
    void waitForWriteResponse(int timeout)
    {
        writeSemaphore.wait(timeout);
    }

    void writeData(ByteBuffer data)
    {
        // An instance of "BLEChar" is usually retrieved via the service and
        // characteristic discovery phase after a connection is established
        BLEChar writeChar = BLEChar("0000fff2-0000-1000-8000-00805f9b34fb");

        for (int idx = 0; idx < data.length; idx += MTU)
        {
            // Set the chunk size to the MTU size or the remaining amount
            int chunk_size = data.length - index;
            if (chunk_size > MTU)
                chunk_size = MTU;

            // void writeCharacteristic(BLEChar writeChar, uint8_t* buffer, size_t length);
            BLE.writeCharacteristic(writeChar, &data[idx], chunk_size);
            waitForWriteResponse();
        }
    }
};
```

Reading

BLE supports two styles of reading data:

- Pull model that requires the application to query the read characteristic for new data.
- Push model that asynchronously notifies the application when new data is available.

The OBDLink CX adapter uses the push model, so you must buffer the incoming data. We recommend using a thread-safe queue so that you can continue to handle received data while processing data on the application side.

We also recommend that you use a concurrent queue of byte buffers, in addition to a normal byte buffer. The concurrent queue of byte buffers handles the incoming data, which comes in as variable sized byte arrays, and the singular byte buffer allows the application side to operate at the byte by byte level.

```
class MyBleHandler: BLEHandler
{
    ConcurrentQueue<ByteBuffer> readQueue;
    ByteBuffer readBuffer;
    Semaphore readSemaphore;
public:
    void notificationReceived(BLEChar notifyChar, ByteBuffer data) override
    {
        if (notifyChar.UUID == "0000fff1-0000-1000-8000-00805f9b34fb")
        {
            readQueue.enqueue(data);
            readSemaphore.signal();
        }
    }

    uint8_t readByte(int timeout)
    {
        if (readBuffer.length == 0)
        {
            // This wait should throw an exception on timeout
            // Or it can return an error and you should handle it appropriately
            readSemaphore.wait(timeout);
            ByteBuffer buffer = readQueue.dequeue();
            readBuffer.append(buffer);
        }

        // Remove the byte at the front of the buffer and return it
        return readBuffer.popFront();
    }
};
```

Learn more

- For a list of useful references when developing an OBD application, refer to this [pinned post \(https://www.scantool.net/forum/index.php?topic=14767.0\)](https://www.scantool.net/forum/index.php?topic=14767.0) in our Support Forum.
- You can explore other OBDLink features and functionality using our support article [knowledge base \(https://support.obdlink.com/en/support/solutions\)](https://support.obdlink.com/en/support/solutions).

- For descriptions of terms used in this article, see the [Glossary of Acronyms and Terms](https://support.obdlink.com/support/solutions/articles/43000723764-glossary-of-acronyms-and-terms) (<https://support.obdlink.com/support/solutions/articles/43000723764-glossary-of-acronyms-and-terms>).

Need more help?

[OBDLink Support Forum](https://www.scantool.net/forum/index.php) (<https://www.scantool.net/forum/index.php>)

[Submit a Support Ticket](https://support.obdlink.com/support/tickets/new) (<https://support.obdlink.com/support/tickets/new>)

OBDLink® is a registered trademark of OBD Solutions, LLC.